

L2 Knowledge Graph - Implementation Summary

EntityMap dimension: graph, API, interactive query UI & docs

Layer L2 - Knowledge Graph | PostgreSQL 16 + Apache AGE 1.6.0

Prototype Jun 10 - Prod Jun 15 - Doc due Jun 12 (async review)

1. What was delivered

The L2 Knowledge Graph's first dimension - the EntityMap - was built end to end on PostgreSQL 16 + Apache AGE, exposed through a REST API, and made visible through an interactive graph page in the existing web app. The other two dimensions (OpsMap, OrgMap) are specified as a baseline plan for review.

- > **Graph backend** - Apache AGE running inside the same Postgres as the crew table.
- > **EntityMap** - 5 entity types + 7 relationship types, built from 40 crew records.
- > **REST API** - 5 endpoints to summarise, search, traverse and visualise the graph.
- > **Query UI** - a new 'Graph' page: filter by rank/cert/port and see the live graph.
- > **Docs** - a baseline design doc, two explainer PDFs, and graph images.

2. Infrastructure stood up

Component	What it is	Where
crew-postgres	Postgres 16 + Apache AGE 1.6.0 (crew table + 'maritime' graph)	localhost:5434
crew-redis	Redis cache for crew lists	localhost:6379
Backend	FastAPI app (GRAPH_BACKEND=age)	localhost:8000
Frontend	Next.js app with the new Graph page	localhost:3000

Switching the database image to AGE lets the relational crew table and the graph live in ONE Postgres - graph context and crew rows are reachable in a single connection.

3. The EntityMap graph

Five entity types, connected by seven relationship types, all keyed on business identity (MERGE) so nothing is duplicated.

Nodes (count)	Relationships
Crew (40), Vessel (5), Port (9), Certificate (16), Contract (20)	HOLDS, ASSIGNED_TO, CURRENTLY_AT, CALLS_AT, SIGNED, FOR_VESSEL, AT_PORT

Totals: 90 nodes, 189 relationships. Because every node is merged on its key, a vessel or port is a

single node no matter how many crew point at it (no duplication).

4. Code added

File	Purpose
backend/database/entity_map.py	EntityMap schema, builder, and queries (search, traverse, subgraph, facets)
backend/scripts/seed_entity_map.py	Loads the EntityMap from the crew table
backend/api/routes/graph.py	The /api/v1/graph REST endpoints
backend/database/graph_db.py	Shared Cypher access - fixed 3 bugs (see s.7)
frontend/src/app/graph/page.tsx	The Graph query page (filters + render)
frontend/src/components/graph/EntityGraph.tsx	React-Flow graph renderer
frontend/src/lib/api.ts	graphApi client + types

5. API endpoints (under /api/v1/graph)

Endpoint	Returns
GET /summary	Per-type node & edge counts (population check)
GET /facets	Distinct ranks / certificates / ports for the UI dropdowns
GET /crew/search	Crew matching rank + certificate + port (any subset)
GET /crew/{id}/traverse	Full multi-hop relationship walk for one crew
GET /subgraph	React-Flow-ready nodes + edges for the query UI

6. The Graph query UI

- > **Where** - a new 'Graph' tab in the app's top navigation (all pages).
- > **Filters** - pick any mix of Rank, Certificate, Port, then Search.
- > **Result** - matched crew and their direct relationships render as an interactive, draggable, zoomable node-graph, colour-coded by entity type.
- > **Live stats** - shows crew / node / edge counts and query time (typically 5-7 ms).
- > **Note** - the view is capped (default 14 crew) for legibility; filtering narrows it. Multi-hop traversal is available via the API; clickable in-UI expansion is a planned add-on.

7. Bugs fixed along the way

- > **Colon clash** - Cypher's [:HOLDS] was mis-read as a SQL parameter; switched to raw SQL.
- > **Unparseable vertices** - AGE tags vertices in a way that breaks JSON; queries now return plain field maps.
- > **Lost writes** - graph writes weren't committed; added a commit so MERGEs persist.
- > **Stale cache** - Redis served a pre-seed empty crew list; flushed the stale keys.
- > **Missing deps** - added greenlet (async DB) and fpdf2/networkx/matplotlib (docs & images).

8. Exit criteria - status

Criterion (from the plan)	Status
All 3 dimensions populated	EntityMap built; OpsMap/OrgMap specified
Crew search by rank + cert + port	Done - correct results
Full relationship traversal works	Done - via traverse API (multi-hop)
Test UI query < 3 seconds	Done - 5-37 ms measured
No data duplication across dimensions	Done - single node per real-world thing
20+ crew test records loaded	Done - 40 loaded

9. Deliverables produced

- > **docs/L2_KNOWLEDGE_GRAPH_DESIGN.md** - baseline design & test doc.
- > **L2_understanding.pdf** - what the L2 layer is, per the plan.
- > **L2ImplementationSimpleWords.pdf** - the build explained in plain language.
- > **L2_entitymap_overview.png / _vessel.png** - rendered pictures of the graph.
- > **This document** - the implementation summary.

10. How to run

```
cd backend
python -m scripts.seed_crew          # 40 crew rows into Postgres
python -m scripts.seed_entity_map    # build the EntityMap graph
uvicorn main:app --port 8000          # backend API
# frontend: cd frontend && npm run dev -> open http://localhost:3000/graph
```

11. Suggested next steps

- > **Click-to-expand** in the UI so multi-hop traversal is visible interactively.
- > **Build OpsMap** - overlay the sign-off -> search -> match -> onboard state on crew nodes.
- > **Build OrgMap** - add Company/Fleet above vessels; turn rank into a shared node.